5. ビルド・保守マニュアル

QCalc をリポジトリのソースから作り直す手順（テスト用 APK と Google Play 用 App Bundle）、Google Play への公開と更新、保守で気をつける点を、開発者向けに説明します。

5.1 開発環境

スクリプトでのビルドは Ubuntu 24.04（64 ビット x86-64）で確認しています（確認環境：Ubuntu 24.04.5 LTS）。Android NDK は使いません。同じソースは Android Studio（Gradle）でも開けるようにしてあります（5.11）。

```
sudo apt install build-essential autoconf automake libtool intltool autopoint
gettext pkg-config \
  aapt dalvik-exchange zipalign apksigner openjdk-17-jdk-headless git curl
xz-utils qemu-user-static
pip install ziglang==0.17.1
# Android SDK の platform ファイル（API 36 と API 34）。Android Studio の SDK
Manager か、次のコマンドで入れる
sdkmanager "platforms;android-36" "platforms;android-34"
# アイコンを作り直すときだけ
sudo apt install librsvg2-bin imagemagick
```

道具	確認した版	使い道
zig（pip の ziglang）	0.17.1	C/C++ を aarch64-linux-musl 向けにクロスコンパイル
aapt2（Ubuntu の aapt パッケージ）	1:14~beta1（aapt2 2.19）	リソースとマニフェストの処理。APK 用と、App Bundle 用の protobuf 形式
android.jar（Android SDK Platform 36）	API 36	Java のコンパイル。場所は $ANDROID_HOME/platforms/android-36、または環境変数 ANDROID_JAR
android.jar（Android SDK Platform 34）	API 34	aapt2 のリンク。Ubuntu の aapt2 は API 35 以降のリソース表を読めないため。環境変数 AAPT2_JAR でも指定できる
JDK（javac・jarsigner・keytool）	OpenJDK 21.0.12（javac は 17 でも確認）	Java 8 の書き方でコンパイル（-source 8 -target 8）、App Bundle の署名、鍵の作成
dalvik-exchange（dx）	10.0.0	class ファイルを classes.dex に変換
zipalign、apksigner	10.0.0、31.0.2	テスト用 APK の整列と署名

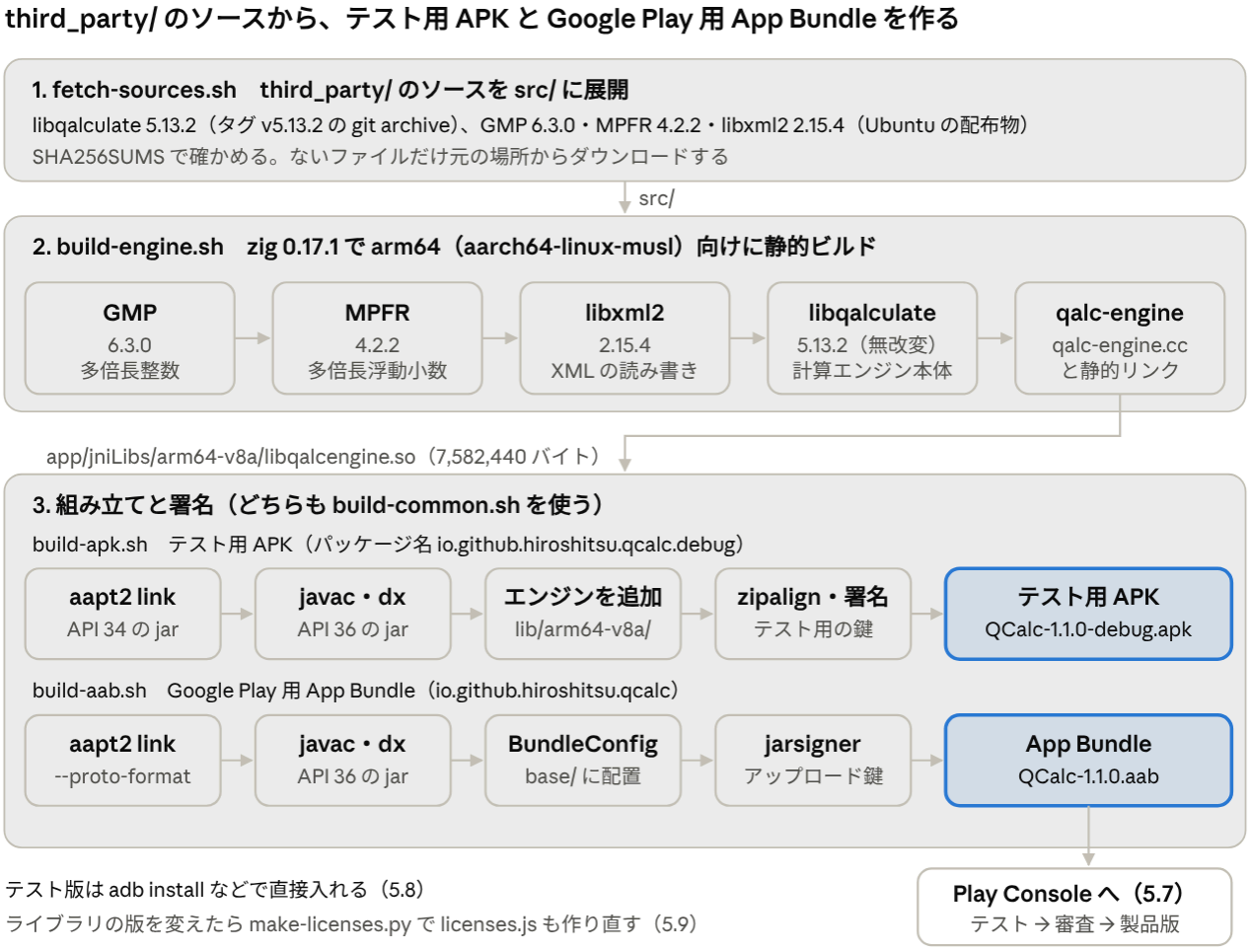
道具	確認した版	使い道
python3	3.13（標準ライブラリだけを使う）	App Bundle の組み立て（build-aab.sh）、ライセンス文の作成（make-licenses.py）
rsvg-convert、ImageMagick	—	アイコンを作り直すとき（icon/make-icons.sh）だけ使う。できたアイコンは app/res と store/ に入っている
qemu-aarch64-static	8.2.2	arm64 のエンジンを PC で動かして試験する
bundletool（任意）	1.17.2	App Bundle の検査と、端末向け APK を作れるかの確認（5.6）
Android Studio（任意）	Android Gradle Plugin 8.11.1・Gradle 8.13 に対応する版	同じソースからビルドし実機で動かす（5.11）。計算エンジンは作らない

フォルダ構成

フォルダ・ファイル	中身	リポジトリに含むか
app/	AndroidManifest.xml、Gradle の設定（build.gradle）、Java（src/io/github/hiroshitsu/qcalc/）、リソース（res/。テスト版の名前は res-debug/）、画面（assets/。licenses.js を含む）、ビルド済みの計算エンジン（jniLibs/arm64-v8a/libqalcengine.so）	含む
bridge/	qalc-engine.cc（エンジンのプログラム）。ビルドすると qalc-engine ができ、app/jniLibs にもコピーされる	qalc-engine.cc だけ含む
third_party/	エンジンに組み込むライブラリのソース（libqalculate・GMP・MPFR・libxml2 の tar.xz）、SHA256SUMS、README.md	含む（GPL・LGPL の対応ソース）
icon/	アイコンの元（qcalc-icon.svg）と、app/res のアイコンと store/icon-512.png を作る make-icons.sh	含む

フォルダ・ファイル	中身	リポジトリに含むか
fetch-sources.sh、build-engine.sh、build-common.sh、build-apk.sh、build-aab.sh、make-licenses.py、version.properties	ビルドのスクリプトと、アプリの版（すべてのビルドで共通）	含む
settings.gradle、build.gradle、gradle.properties、gradle/wrapper/、keystore.properties.example	Android Studio（Gradle）用の設定と、署名の設定の見本	含む
docs/	GitHub Pages で公開するページ（紹介ページ、プライバシーポリシー）	含む
store/	Google Play の掲載文（listing-ja.md）、アイコン、フィーチャーグラフィック、スクリーンショット	含む
README.md、LICENSE、.gitignore	説明、GNU GPL v2 の全文、git で管理しないものの一覧	含む
src/	third_party/ から展開したソース	含まない（スクリプトで作る）
tools/、build/、prefix/、sdk/、Gradle の作業フォルダ（.gradle/、build-gradle/、app/build/）	zig の呼び出し用スクリプト、作業用フォルダ、静的ライブラリの置き場、Android Studio のビルド結果	含まない
out/	できあがった APK と App Bundle	含まない
signing/、keystore.properties	アップロード鍵（qcalc-upload.jks）とそのパスワードのファイル、テスト用の鍵（qcalc-debug.keystore）、Android Studio で署名するときの設定	含めない（.gitignore で除外し、別に保管する）

5.2 ビルドの流れ



ビルドの流れ · 3 段階でテスト用 APK と App Bundle

3 段階のスクリプトで、テスト用 APK と Google Play 用 App Bundle を作ります。build-engine.sh は作り終えたライブラリ（prefix/lib の .a）を飛ばすので、画面や Java だけを直したときは build-apk.sh か build-aab.sh だけ、qalc-engine.cc を直したときは build-engine.sh から実行します。リポジトリにはビルド済みの計算エンジン（app/jniLibs）が入っているので、エンジンを変えないなら zig も fetch-sources.sh も不要です。Android Studio でのビルドは 5.11 にあります。

```
./fetch-sources.sh        # third_party/ のソースを src/ に展開（最初の 1 回）
./build-engine.sh         # ライブラリとエンジンをビルドし、app/jniLibs にコピー
./build-apk.sh            # テスト用 APK：out/QCalc-1.1.0-debug.apk
./build-aab.sh            # Google Play 用：out/QCalc-1.1.0.aab（アップロード鍵が必要）
python3 make-licenses.py  # ライブラリの版を変えたときだけ：app/assets/licenses.js
```

5.3 ソースの準備（fetch-sources.sh と third_party/）

ソース	版	third_party/ のファイル	元の入手先
libqalculate	5.13.2	libqalculate-5.13.2.tar.xz	https://github.com/Qalculate/libqalculate のタグ v5.13.2（コミット d3672af9cf6e05a39bd4206b7b7d6b1e0d194caa）を git archive で固めたもの
GMP	6.3.0	gmp_6.3.0+dfsg.orig.tar.xz	https://archive.ubuntu.com/ubuntu/pool/main/g/gmp/
MPFR	4.2.2	mpfr4_4.2.2.orig.tar.xz	https://archive.ubuntu.com/ubuntu/pool/main/m/mpfr4/
libxml2	2.15.4	libxml2_2.15.4+dfsg.orig.tar.xz	https://archive.ubuntu.com/ubuntu/pool/main/libx/libxml2/

- third_party/ にある tar.xz を src/ に展開します。ないファイルだけ元の入手先からダウンロードします（libqalculate はタグ v5.13.2 を git clone して git archive で固める）。
- Ubuntu から入手した 3 つは SHA256SUMS（sha256sum -c）で中身を確かめ、合わなければ止まります。
- third_party/ のソースは、GPL・LGPL が求める「対応するソース」としてリポジトリで公開するためのものです。版を変えたら、ファイルと SHA256SUMS、README.md も入れ替えます（5.9）。
- Ubuntu が配布している GMP のソースには説明書のフォルダがないため、configure が必要とする doc/Makefile.in を中身のないものとして作ります。
- 上流の v5.13.2 タグの configure.ac は版番号が 5.13.1 のままです（qalc --version も 5.13.1 と表示）。アプリが表示する版は、build-engine.sh が渡す QALC_ENGINE_VERSION（"5.13.2"）です。

5.4 計算エンジンのビルド（build-engine.sh）

zig をクロスコンパイラとして使い、すべてを静的にリンクした arm64 の実行ファイルを作ります。Android のライブラリに依存しないので、Android NDK は不要です。

1. tools/ に zig の呼び出し用スクリプト（zcc、zcxx、zar、zranlib、対象は aarch64-linux-musl）を作る。
2. GMP → MPFR → libxml2 → libqalculate の順に、静的ライブラリ（-O2）として prefix/ に入れる。prefix/lib に .a があるライブラリは飛ばすので、作り直すときはその .a を消す。
3. bridge/qalc-engine.cc を C++17 でコンパイルし、全部を静的にリンクする（-static、-s で記号を削除、-Wl,-z,max-page-size=16384 で 16 KB ページの端末に対応）。できるのは bridge/qalc-engine（7,582,440 バイト。版 1.1.0）で、APK と App Bundle に入れるため

に app/jniLibs/arm64-v8a/libqalcengine.so にもコピーする。

libqalculate の configure オプション

オプション	理由
--enable-compiled-definitions	単位・関数などの定義をプログラムに埋め込む（定義ファイルを配布しなくてよい）
--disable-insecure	コマンドを実行する command() と uptime を無効にする
--without-libcurl	通信機能を入れない（為替レートはアプリの Java が取得する）
--without-icu	ICU を使わない
--without-gnuplot-call	gnuplot を呼ばない（グラフは画面が描く）
--with-readline=no	対話用の行編集は不要
--disable-nls	翻訳を入れない（メッセージは英語。よく出るものは画面が日本語にする）

libxml2 は python・zlib・lzma・ICU・HTTP・デバッグ・カタログを外してビルドします。各工程のログは build/〈ライブラリ名〉/ の configure.log、make.log に残り、失敗したときは末尾が表示されます。

5.5 テスト用 APK の作成（build-apk.sh）

1. リソース（app/res）：アプリ名（QCalc）、明るいテーマと暗いテーマ（values-night）、Android 8.0 以降のアダプティブアイコン（テーマアイコン用のモノクロを含む）。テスト版では app/res-debug がアプリ名を「QCalc (debug)」に変える。
2. マニフェストに package 属性（Java のパッケージと同じ io.github.hiroshitsu.qcalc）を補い、aapt2 でリソースをコンパイルして、マニフェストと画面（app/assets）を含む APK の土台を作る。リンクのときに --rename-manifest-package でパッケージ名を io.github.hiroshitsu.qcalc.debug に変える。aapt2 はこのとき .MainActivity を io.github.hiroshitsu.qcalc.MainActivity という完全な名前に書き換える（Gradle の applicationIdSuffix と同じ動き）。マニフェストに直接 io.github.hiroshitsu.qcalc.debug と書くと、Android は存在しない io.github.hiroshitsu.qcalc.debug.MainActivity を探すため、アプリが起動しない。リンクのあとに aapt2 dump badging で、起動するアクティビティが io.github.hiroshitsu.qcalc.MainActivity であることを確かめ、違えば止まる。リンクには API 34 の android.jar を使う。minSdk 24、targetSdk 36、versionCode・

versionName は version.properties から読み、versionName には -debug を付ける。ソースのマニフェストに package 属性がないのは、Gradle がパッケージ名を namespace で決めるため。

3. Java を API 36 の android.jar でコンパイルし（-source 8 -target 8）、dx で classes.dex にする（build-common.sh の build_dex。App Bundle と共通）。
4. エンジン（app/jniLibs/arm64-v8a/libqalcengine.so）を lib/arm64-v8a/libqalcengine.so として入れる。Android は lib/ の lib〜.so だけを実行できる場所（nativeLibraryDir）に展開するためで、マニフェストの extractNativeLibs="true" と組み合わせて使う。
5. zipalign（4 バイト境界、-p）してから apksigner で署名する（v1・v2・v3）。できるのは out/QCalc-1.1.0-debug.apk（3,295,316 バイト。版 1.1.0）。

テスト用の鍵

- signing/qcalc-debug.keystore（別名 debug）。なければ最初のビルドで keytool が作る（RSA 3072 ビット、有効期間 10,000 日、CN=QCalc debug）。パスワードは build-apk.sh に書かれている（テスト版専用の鍵のため）。
- テスト版にだけ使う。Google Play 版とはパッケージ名も鍵も別なので、並べて入れられ、互いに上書きしない。
- 鍵を作り直すと、入っているテスト版には上書きできなくなる（テスト版をアンインストールしてから入れる）。

確認のコマンド

```
apksigner verify --verbose --print-certs out/QCalc-1.1.0-debug.apk   # 署名と証明書
aapt2 dump badging out/QCalc-1.1.0-debug.apk                         # パッケージ名・版・SDK・権限・起動するアクティビティ
unzip -l out/QCalc-1.1.0-debug.apk                                   # 中身（lib/arm64-v8a/libqalcengine.so など）
```

5.6 Google Play 用 App Bundle の作成（build-aab.sh）

Google Play には App Bundle（.aab）をアップロードします。build-aab.sh は Gradle や bundletool を使わずに .aab を組み立て、アップロード鍵で署名します（中身は詳細設計書 3.17）。

1. マニフェストに package 属性（io.github.hiroshitsu.qcalc）を補い、aapt2 link --proto-format で protobuf 形式のマニフェストとリソース表（resources.pb）を作る（API 34 の android.jar。minSdk 24、targetSdk 36、版は version.properties）。
2. Java を API 36 の android.jar でコンパイルし、classes.dex にする（テスト用 APK と同じ）。

3. BundleConfig.pb（bundletool 1.17.2、「ネイティブライブラリを圧縮しない」最適化を無効）と、base/manifest・base/dex・base/res・base/assets・base/lib/arm64-v8a/libqalcengine.so を ZIP にまとめる（各項目の時刻は 1981-01-01 に固定）。
4. jarsigner（SHA-256、SHA256withRSA）でアップロード鍵の署名を付け、jarsigner -verify で確かめる。できるのは out/QCalc-1.1.0.aab（3,282,686 バイト。版 1.1.0 の SHA-256 は 87afbfd8b19bca85c644f13594e1ff4543805992166459d7418240d89ab73e5e）。

アップロード鍵

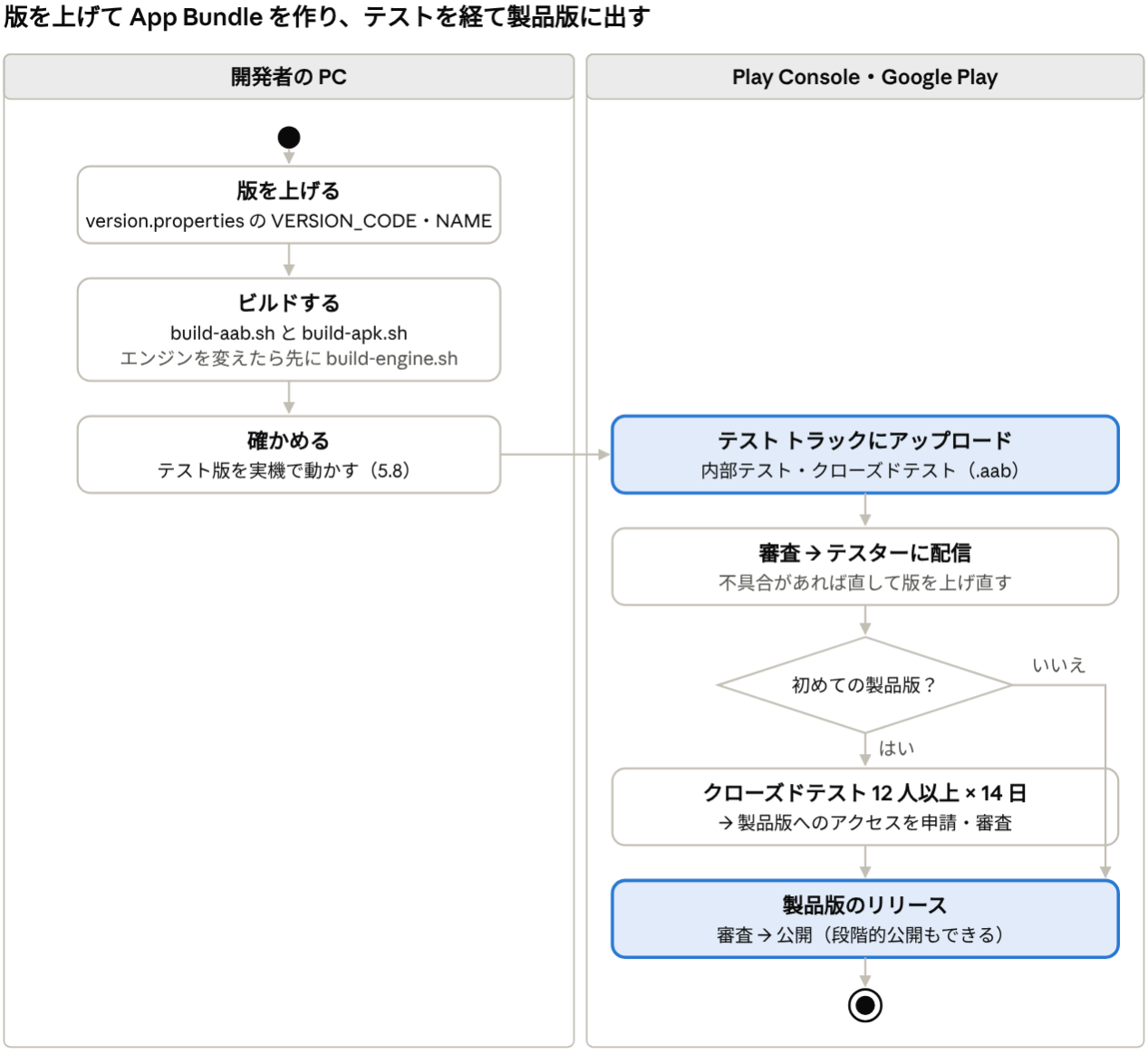
- signing/qcalc-upload.jks（別名 upload）。パスワードは環境変数 QCALC_UPLOAD_PASS か、ファイル signing/qcalc-upload.pass の 1 行目から読む。場所と別名は QCALC_UPLOAD_KEYSTORE・QCALC_UPLOAD_ALIAS でも変えられる。
- Google Play アプリ署名を使うので、利用者の端末に届く APK には Google が管理するアプリ署名鍵の署名が付く。アップロード鍵は、開発者からのアップロードであることを Google Play が確かめるためだけに使う。
- 鍵とパスワードは公開しない（signing/ は .gitignore で除外）。控えを安全な場所に保管する。なくしたり漏れたりしたときは、Play Console からアップロード鍵の再設定を申請する（新しい鍵が登録されるまで更新を出せない）。

確認のコマンド（bundletool は https://github.com/google/bundletool/releases から）

```
jarsigner -verify out/QCalc-1.1.0.aab                                  # 署名を確かめる
keytool -printcert -jarfile out/QCalc-1.1.0.aab                        # アップロード鍵の証明書（Play Console に登録したものと同じか）
java -jar bundletool.jar validate --bundle=out/QCalc-1.1.0.aab
java -jar bundletool.jar dump config --bundle=out/QCalc-1.1.0.aab      # uncompressNativeLibraries が有効でないこと
java -jar bundletool.jar build-apks --bundle=out/QCalc-1.1.0.aab --output=qcalc.apks --device-spec=phone.json
```

5.7 Google Play への公開と更新

初めての公開の詳しい手順（Play Console の入力項目、データセーフティ、テスターの集め方など）は `QCalc を Google Play で公開する手順` にあります。ここでは流れと、版を更新するときの手順をまとめます。



アクティビティ図 · 公開と更新の流れ

版を更新するとき

1. version.properties の VERSION_CODE を前の版より大きくし（例：2 → 3。版 1.0.0 は 1、版 1.1.0 は 2）、VERSION_NAME を変える（例：1.1.1）。
2. 画面・Java だけの変更なら ./build-aab.sh、エンジンを変えたら ./build-engine.sh のあとで ./build-aab.sh を実行する。
3. ./build-apk.sh でテスト版も作り、実機で 5.8 の確認をする。
4. Play Console でトラック（内部テスト・クローズドテスト・製品版）を選んで新しいリリースを作り、.aab をアップロードしてリリースノートを書き、審査に送る。
5. 審査が通ると配信される。製品版では段階的公開（配信する割合を少しずつ増やす）も使える。

- VERSION_CODE は一度使った値を二度と使えない（同じ値の .aab は受け付けられない）。アップロード鍵は毎回同じものを使う。

- 新しい個人のデベロッパー アカウントは、製品版を申請する前に、12 人以上のテスターが 14 日間続けて参加したクローズドテストが必要（2026 年 11 月時点の Google Play の条件）。
- targetSdk は Google Play の条件（毎年 8 月 31 日から、新しい API レベルが求められる）に合わせて上げる。QCalc 1.1.0 は 36。
- 機能や通信先を変えたら、プライバシーポリシー（docs/privacy.html）、データセーフティの申告、ストアの掲載文（store/listing-ja.md）も見直す。リポジトリのソースも、配布した版と同じものに更新する（GPL）。

5.8 動作確認の方法

エンジンだけを PC で動かす

arm64 のエンジンを qemu で動かし、JSON を 1 行ずつ渡します。

```
echo '{"id":1,"cmd":"calc","expr":"integrate(x^2 sin(x), 0, pi)"}' |
QALCULATE_USER_DIR=/tmp/q qemu-aarch64-static bridge/qalc-engine
echo '{"id":2,"cmd":"plot3d","exprs":["sin(x) cos(y)"],"nx":5,"ny":5}' |
QALCULATE_USER_DIR=/tmp/q qemu-aarch64-static bridge/qalc-engine
echo '{"id":3,"cmd":"calc","expr":"[1, 2; 3, 4] × [5, 6; 7, 8]"}' |
QALCULATE_USER_DIR=/tmp/q qemu-aarch64-static bridge/qalc-engine
```

結果は「"result":"π² − 4 ≈ 5.869604401"」を含む 1 行です。"store":true を付けると、続けて ans イベントの行も出ます。plot3d は x・y・series（式ごとの z）を含む 1 行を返します。行列は「"result":"[19 22; 43 50]"」のように 1 行の文字で返り（行の区切りは「; 」）、画面が表の形にします。公式の qalc との比較は、同じタグの libqalculate を PC 向けにビルドして qalc -defaults で計算し、結果を並べます（6.2）。

画面を PC で動かす

開発中は、app/assets を配信してエンジンへの要求を qemu のエンジンに中継する小さな HTTP サーバーを使いました。ページに window.QalcDev（send を POST /engine に転送）を差し込み、Chromium（Playwright）で表示して自動操作します。為替レートの更新はこのサーバーでは模擬です。操作マニュアルの画像と store/ のスクリーンショットもこの道具で撮っています。この試験用の道具はリポジトリには含めていません。

実機で確かめる

```
adb install -r out/QCalc-1.1.0-debug.apk
adb logcat -s QCalc      # 画面の JavaScript のエラーなど（MainActivity がタグ
QCalc で出力）
```

確かめること：起動と計算、キーの 4 つのタブ（123・関数・解析・行列）とひな形の入力、行列の結果の表の形、式の例、元に戻す、文字の大きさ、履歴の再利用、アプリを閉じて開き直したときに履歴・入力中の式・ans が残ること、為替レートの更新、2D・3D グラフ（移動・回転・拡大・タップ）、ユーザー定義、ダークモードの切り替え、画面の回転、戻る操作（ジェスチャーと予測型「戻る」）、Android 15 以降でステータスバー・ナビゲーションバー・キーボードと画面が重ならないこと、「このアプリについて」とオープンソースライセンス、TalkBack。Google Play から配る APK での動きは、内部テストに .aab を上げて Google Play から入れると確かめられます。

5.9 新しい libqalculate への更新手順

1. 上流の新しいタグ（https://github.com/Qalculate/libqalculate）と変更内容を確かめる。
2. third_party/ に新しいソース（タグを git archive で固めた tar.xz）を置き、古いものを消す。third_party/README.md の版、タグ、コミットも書き換える。
3. 版番号を書き換える（下の表）。
4. prefix/lib/libqalculate.a と src/ の古いフォルダを消してから、./fetch-sources.sh と ./build-engine.sh を実行する。
5. qalc-engine.cc のコンパイルエラーと警告を直す。calculate_once と store_ans は qalc の計算手順（Calculator::calculateAndPrint と qalc.cc）を写したものなので、上流でそれらが変わっていないか差分を確かめて反映する。plot・plot3d が使う expressionToPlotVector・generateVector の動きも確かめる。日本の単位と trace()（add_japanese_units、add_matrix_functions）は、同じ名前の単位や関数が上流に加わっていれば追加しないので、どちらが使われるかを確かめる。メッセージの保留を閉じる処理（close_message_stops）は、上流で問題が直っていてもそのままでよいが、試験 M03〜M05 で警告が出ることを確かめる（詳細設計書 3.19）。
6. 為替レートの取得元やファイル形式が変わっていないかを getExchangeRatesUrl・getExchangeRatesFileName で確かめ、必要なら RatesUpdater の検査を直す。
7. 新しい分類名やメッセージがあれば app.js の CATEGORY_JA、MESSAGES_JA、HIDDEN_ITEMS を直す。新しい関数・定数・単位の日本語の名前は FUNC_JA・VAR_JA・JA_NAMES に足す。式の例（EXAMPLES）は試験で全部をエンジンに通し、結果が変わっていないか確かめる。
8. python3 make-licenses.py でライセンス文を作り直す（著作権の年が変わっていれば make-licenses.py の copyright も直す）。
9. ./build-apk.sh と ./build-aab.sh で作り、6 章の試験をやり直す（公式 qalc との比較を含む）。Google Play には 5.7 の手順で出す。

ファイル	書き換えるところ
fetch-sources.sh	git clone のタグ（--branch v5.13.2）、ファイル名（libqalculate-5.13.2.tar.xz）、フォルダ名
third_party/README.md	版、タグ、コミット
build-engine.sh	src/libqalculate-5.13.2 のフォルダ名（2 か所）と -DQALC_ENGINE_VERSION='"5.13.2"'
make-licenses.py	libqalculate のフォルダ名と版（'5.13.2'）
version.properties	VERSION_NAME（例：1.1.1）と VERSION_CODE（必ず前より大きくする）
app/src/io/github/hiroshitsu/qcalc/RatesUpdater.java	User-Agent の libqalculate の版（「QCalc/〈アプリの版〉 (Android; libqalculate 5.13.2)」。アプリの版は versionName から自動で入る）
README.md	版番号（libqalculate 5.13.2）

GMP・MPFR・libxml2 を更新するときは、third_party/ のファイルと SHA256SUMS、fetch-sources.sh の URL とファイル名、build-engine.sh のフォルダ名、make-licenses.py の版とフォルダ名を書き換え、prefix/lib の該当する .a と、それに依存するライブラリ（最後には libqalculate.a）を消してからビルドします。

5.10 保守上の注意

項目	注意すること
アップロード鍵	公開しない。控えを安全な場所に取る。なくしたり漏れたりしたら Play Console から再設定を申請する。アプリ署名鍵は Google が管理しているので、アップロード鍵をなくしても利用者のアプリは更新し続けられる
versionCode	出すたびに大きくする（version.properties の VERSION_CODE）。一度使った値は使えない

項目	注意すること
targetSdk	36。Google Play の条件に合わせて毎年上げる。上げたら、その版の動作の変更（画面の縁、戻る操作など。詳細設計書 3.15）を確かめる
エンジンのファイル名と展開	lib/arm64-v8a/libqalcengine.so という名前、extractNativeLibs="true"、App Bundle の uncompressNativeLibraries を無効にする設定（Gradle では useLegacyPackaging = true）は変えない。Android 10 以降はアプリのデータフォルダのファイルを実行できないため、nativeLibraryDir に展開される形で入れている
16 KB ページ	エンジンのリンクの -Wl,-z,max-page-size=16384 を外さない（Google Play の条件）
android.jar の使い分け	Java は API 36、aapt2 は API 34 の android.jar を使う。新しい API は Build.VERSION.SDK_INT を確かめてから使う（minSdk は 24）
画面と Java の境界	QalcNative に公開するのは 8 つのメソッドだけ。追加するときも引数を検査する（StateStore の名前、URL の種類など）。WebView のファイルアクセスは無効のままにする
為替レートの取得元	URL とファイル名は libqalculate から受け取るが、検査の条件と 3 番目の予備の URL は RatesUpdater に書いてある。取得元の形式が変わったら直す
libqalculate に足しているもの	日本の単位 10 個、trace()、メッセージの保留を閉じる処理は、libqalculate のソースではなく qalc-engine.cc にある（詳細設計書 3.19、README の「libqalculate との違い」）。libqalculate のソースは変えない方針を続け、結果が公式の qalc と同じであることを比較の試験で確かめる
ライセンス	アプリは GPL v2 以降、エンジン全体は GPL v3 以降（MPFR が LGPL v3 以降のため）。ライブラリを変えたら third_party/ と licenses.js も変え、配布している版と同じソースをリポジトリで公開し続ける
プライバシー	個人情報を集めない設計。解析・広告・新しい通信先を加えるときは、プライバシーポリシー（docs/privacy.html）、アプリ内の説明、データセーフティの申告を先に直す

項目	注意すること
試験の道具	開発で使った試験用のスクリプト（6 章）はリポジトリに含まれていない。保守で繰り返し使うなら、加えるとよい

5.11 Android Studio で開く

同じソースを Android Studio（Gradle）で開き、ビルドと実機での実行ができるように設定を入れています。計算エンジンは作らず、app/jniLibs のビルド済みのものをそのまま入れます。

項目	内容
必要なもの	Android Gradle Plugin 8.11.1 に対応した Android Studio。Gradle 8.13 と Android SDK Platform 36 は、初めて開いたときに案内に従ってダウンロードする
開き方	「Open」でリポジトリの一番上のフォルダ（settings.gradle があるところ）を選ぶ。同期が終わると app モジュールが表示される
ソースの場所	app/build.gradle の sourceSets で、build-apk.sh と同じフォルダ（app/src、app/res、app/assets、app/jniLibs）を指している。版は version.properties から読む
デバッグ版	実行（▶）で入る。パッケージ名は io.github.hiroshitsu.qcalc.debug、名前は「QCalc (debug)」（app/res-debug）。Google Play 版とは別のアプリとして入り、履歴や設定も別
Google Play 用	Build メニューの「Generate Signed App Bundle or APK…」で Android App Bundle を選び、アップロード鍵 signing/qcalc-upload.jks（別名 upload）を指定する。keystore.properties（keystore.properties.example をコピーして書く）を置くと、release のビルドに自動で署名される
スクリプトとの違い	compileSdk 36・targetSdk 36 で、dex は D8 で作る。エンジンは圧縮して入れ、インストール時に展開される（useLegacyPackaging = true）

注意すること

- Gradle で作った .aab は、bundletool dump config で uncompressNativeLibraries が有効になっていない（"enabled": true がない）ことを確かめてから出す。有効だと Google Play が作る APK でエンジンが展開されず、計算できない。
- コマンドライン用の gradlew と gradle-wrapper.jar は入っていない。Android Studio では gradle/wrapper/gradle-wrapper.properties があれば足りる。
- qalc-engine.cc や libqalculate を変えたときは、Linux（Windows では WSL）で build-engine.sh を実行して app/jniLibs のエンジンを作り直す。
- この設定は Gradle と Android SDK を取得できない環境で作ったため、Android Studio でのビルドはまだ確かめていない（6 章）。